

Introduction

Dans un contexte industriel marqué par la transformation numérique, l'automatisation des processus et l'exploitation massive des données, les entreprises multinationales ont besoin de solutions performantes leur permettant de collecter, centraliser, analyser et valoriser les informations issues de leurs équipements de production.

bAcTechFlow est une plateforme de gestion, supervision et analyse des données industrielles conçue pour répondre à ces exigences. L'application permet de collecter en temps réel les données générées par les machines de production réparties dans les différentes usines d'une entreprise, quelle que soit leur localisation géographique à travers le monde.

L'architecture organisationnelle prise en charge par bAcTechFlow repose sur plusieurs niveaux hiérarchiques. Une entreprise peut posséder plusieurs usines implantées dans différents pays ou régions. Chaque usine est composée d'un ou plusieurs ateliers spécialisés, eux-mêmes constitués de plusieurs machines ou équipements industriels générant continuellement des données de fonctionnement, de production, de qualité ou de maintenance.

Grâce à une infrastructure de communication sécurisée et distribuée, bAcTechFlow assure la remontée automatique des données depuis les machines jusqu'aux serveurs centraux de l'entreprise selon deux étapes clés :

1. **À l'échelle locale (Usine)** : Une première étape de centralisation intermédiaire est implantée dans l'usine elle-même. Elle présente l'avantage d'alléger le volume de données traitées à la source et d'offrir une vision opérationnelle rapprochée.
2. **À l'échelle globale (Siège)** : Ces données sont ensuite transmises au siège central pour obtenir une vision macroscopique et stratégique.

Les informations ainsi collectées sont stockées, historisées et mises à disposition des différents acteurs de l'organisation (équipes sur site et responsables au siège) sous forme de tableaux de bord, d'indicateurs de performance (KPI), de rapports et d'outils d'analyse avancés.

La plateforme offre ainsi une vision globale et unifiée de l'activité industrielle, permettant aux directeurs d'usines, aux chefs d'ateliers, aux équipes de maintenance et aux décideurs stratégiques de suivre les performances des équipements, d'identifier rapidement les anomalies, d'optimiser les processus de fabrication et d'améliorer la prise de décision. En centralisant les données de l'ensemble des sites au sein d'un système unique, bAcTechFlow constitue un véritable levier de compétitivité, favorisant l'amélioration continue, la réduction des coûts d'exploitation et l'augmentation de la productivité, en parfaite conformité avec les principes de l'**Industrie 4.0**.

Infrastructure

Outre le siège central qui dispose d'un service de contrôle et d'analyse des données, chaque usine est identifiée par le pays et le département où elle est implantée. Une usine type possède un ou plusieurs ateliers, chaque atelier abritant une ou plusieurs machines industrielles.

1. Les machines industrielles

La machine industrielle constitue la mine de données et l'entité de base de bAcTechFlow. Chaque machine communique via un protocole industriel qui peut être traditionnel ou moderne. L'approche retenue dans bAcTechFlow garantit une intégration progressive des installations existantes sans nécessiter le remplacement immédiat des équipements anciens, tout en préparant l'entreprise à la transition vers l'usine connectée.

- **Protocoles industriels traditionnels** : Développés principalement pour la communication locale entre automates programmables (API), capteurs et systèmes SCADA au sein d'un même site. Ils restent très répandus en raison de leur robustesse :
 - *Exemples* : Modbus (RTU/TCP), Profibus, DeviceNet, CANopen, Interbus, HART, DF1, MPI.
 - *Caractéristiques* : Architecture maître-esclave, débits limités, liaisons séries prédominantes (RS-232/RS-485), absence native de mécanismes de cybersécurité, usage strictement local.
- **Protocoles industriels moderne** : Émergents avec l'Industrie 4.0, ils offrent une interopérabilité native avec les réseaux IP, le Edge/Cloud computing et les outils d'analyse avancés.
 - *Exemples* : OPC UA, MQTT, AMQP, DDS, EtherNet/IP, PROFINET, EtherCAT, POWERLINK, TSN.
 - *Caractéristiques* : Communications basées sur Ethernet et TCP/IP, haut débit et faible latence, interopérabilité multi-constructeurs, sécurité intégrée (chiffrement, certificats, authentification), intégration directe avec les systèmes MES, ERP et Cloud.

2. Le collector (Gateway Edge)

Afin de collecter les données des machines, un collecteur (*Gateway Edge*) doit être déployé. Deux choix d'architectures s'offrent aux décideurs selon les contraintes de l'usine :

- **Architecture distribuée (Un collecteur par atelier)** : Le collecteur récupère les données d'une dizaine de machines maximum par écoute directe de port ou par interrogation de registres. Il formate les informations avant de les acheminer vers le serveur de l'usine. **Avantage majeur** : En cas de panne réseau ou électrique du serveur central, le

collecteur assure un stockage local temporaire (*Store and Forward / Buffer*) pour éviter toute perte de données.

- **Architecture centralisée (Un seul collecteur dans la salle de contrôle)** : Plus simple à implémenter physiquement, mais elle impose au collecteur de gérer un nombre massif de machines. Cela présente un risque élevé de saturation et de perte de données en temps réel face à un volume de données trop important.

3. La salle de contrôle

Elle centralise les flux et peut aller d'une configuration basique avec un contrôleur unique à une structure complète animée par une équipe d'analystes. Les utilisateurs y bénéficient de représentations visuelles (tableaux, graphiques en temps réel) et d'outils d'intelligence artificielle pour les statistiques prédictives et discriminantes.

Schéma synoptique du flux des données

Le diagramme suivant détaille la cinétique de traitement et d'acheminement des flux de données, depuis l'acquisition terrain jusqu'à la consolidation au niveau du Cloud d'entreprise.

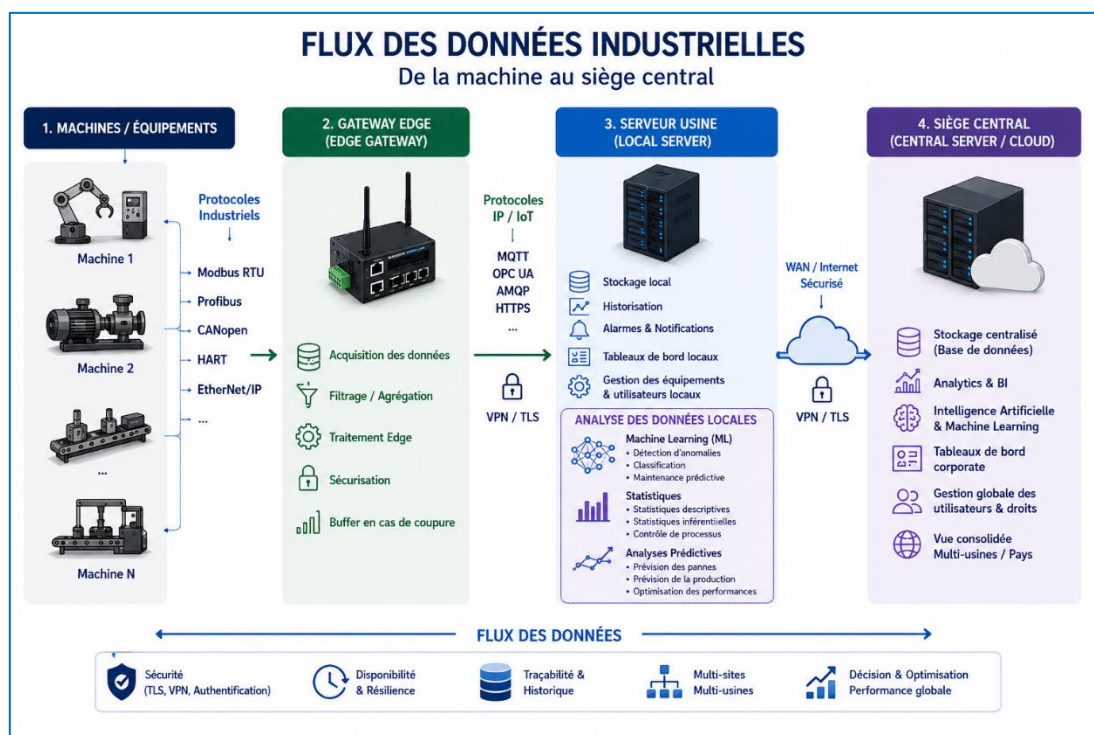


Figure 1 : diagramme de flux des données industrielles.

Partie informatique

Afin d'implanter et de valider l'application bAcTechFlow, une infrastructure de simulation a été mise en place, exploitant un cluster de 4 PC configurés ainsi :

- **PC Xeon** : Serveur principal de la salle de contrôle.
- **PC Mint** : Collecteur Edge (simulé ici en architecture centralisée).
- **PC Surface** : Simulateur des machines de l'**Atelier n°1**.
- **PC Lenovo** : Simulateur des machines de l'**Atelier n°2**.

Pour le déploiement des modules logiciels, deux approches sont étudiées :

- **L'approche standard (Déploiement natif)** : Simple à mettre en œuvre mais dépendante du système d'exploitation hôte. Si l'application plante (erreur critique, coupure réseau), elle s'arrête définitivement. Sa résilience nécessite l'écriture de scripts système (Services Windows ou Daemons Systemd Linux) pour forcer le redémarrage. De plus, la maintenance est lourde car l'équipe informatique doit gérer des livrables différents selon l'OS.
- **L'approche industrielle (Conteneurisation Docker)** : L'exécutable est encapsulé dans une image Docker unique. Le seul prérequis sur le poste cible est le moteur Docker. Elle offre une uniformité totale (la même image tourne sur le PC Mint ou n'importe quel autre PC), une résilience automatique (option `--restart unless-stopped` qui relance le conteneur en arrière-plan en cas de crash), une isolation de sécurité et un déploiement rapide en une seule ligne de commande.

Focus sur l'approche standard

Dans cette section dédiée au déploiement natif, nous nous concentrons sur le fonctionnement du collecteur (PC Mint). Son rôle est d'écouter le topic du broker MQTT, de traiter les messages des ateliers simulés (Surface et Lenovo), d'enregistrer ces données dans une base de données et de les acheminer vers le serveur de la salle de contrôle.

Chaque machine est identifiée de manière unique selon la nomenclature officielle : USINE-uu-ATELIER-aa-MACHINE-mm. Les métriques collectées en temps réel incluent la vitesse de rotation du moteur (`motor_rpm`), la température (`temperature_celsius`), la production cumulée (`total_production_meters`) et l'état de fonctionnement (`status`).

1. Script du Collecteur Edge (Node.js)

Ce script collecte les données depuis les machines en écoutant le topic bAcTechFlow-DataCenter/telemetry.

```
import mqtt from 'mqtt';

const XEON_URL = 'http://192.168.1.200:8080/api/telemetry';

// 1. Connexion au Broker MQTT
const mqttClient = mqtt.connect('mqtt://localhost:1883');
const TOPIC_TELEMETRY = 'bAcTechFlow-DataCenter/telemetry';
mqttClient.on('connect', () => {
  console.log('⚡ Passerelle Edge Atelier connectée au Broker MQTT.');
  mqttClient.subscribe(TOPIC_TELEMETRY, (err) => {
    if (!err) { console.log('📡 Abonnement réussi au topic : ${TOPIC_TELEMETRY}'); }
    else { console.error("Erreur d'abonnement :", err); }
  });
});

// 2. Interception des messages MQTT + envoi HTTP vers le Xeon
mqttClient.on('message', async (topic, message) => {
  try {
    const data = JSON.parse(message.toString());
    const machine_id = data.machine_id;
    const status = data.status;
    const motor_rpm = data.metrics.motor_rpm;
    const temperature_celsius = data.metrics.temperature_celsius;
    const total_production_meters = data.metrics.total_production_meters;

    // Affichage console pour vérification
    console.log(`machine: ${machine_id} rpm: ${motor_rpm} température: ${temperature_celsius}
total production: ${total_production_meters} état: ${status}`);

    // On peut ajouter ici un test de la connexion au serveur Xeon,
    // si connexion OK : Envoi HTTP POST vers le serveur Xeon, sinon enregistrement dans la base
    // locale des données
    const response = await fetch(XEON_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        machineId: machine_id,
        status: status,
        motorRpm: motor_rpm,
        temperatureCelsius: temperature_celsius,
        totalProductionMeters: total_production_meters
      })
    });
  }
});
```

```
    if (!response.ok) {  
        console.error(`✖ Erreur HTTP vers Xeon : ${response.status}`);  
    }  
} catch (error) {  
    console.error("✖ Erreur lors du traitement ou de l'envoi :", error);  
}  
});
```

Pour exécuter ce script sur le collecteur, l'environnement Node.js doit être installé. Lancement via la commande : `npm install mqtt` et ensuite `node nom_du_fichier.js`

2. Script de Simulation des Machines Industrielles (Node.js)

Ce script, exécuté sur les postes de simulation (**PC Surface** pour l'Atelier 1 et **PC Lenovo** pour l'Atelier 2), génère de manière dynamique des charges de données réalistes.

```
JavaScript
import mqtt from 'mqtt';

// Connexion au broker MQTT centralisé (le Gateway Edge = PC Mint)
const client = mqtt.connect('mqtt://192.168.1.115:1883');

// Configuration par variables d'environnement ou valeurs par défaut
const NB_MACHINES = process.env.NB_MACHINES ? parseInt(process.env.NB_MACHINES) : 5;
const ATELIER = process.env.ATELIER || "ATELIER02";

client.on('connect', () => {
  console.log(`🚀 [Lancement] Simulation de ${NB_MACHINES} machines pour l'${ATELIER}...`);
  for (let i = 1; i <= NB_MACHINES; i++) {
    const machineNumber = String(i).padStart(2, '0');
    const MACHINE_ID = `USINE01-${ATELIER}-MACHINE${machineNumber}`;

    // Compteur initial de production propre à CHAQUE machine
    let totalMetersProduced = 12450.0;

    // Intervalle indépendant de 1 seconde pour chaque machine
    setInterval(() => {

      // Génération de métriques aléatoires réalistes
      const rpm = Math.floor(750 + Math.random() * 20);
      const temperature = parseFloat((42.5 + Math.random() * 3).toFixed(1));
      totalMetersProduced += parseFloat((0.2 + Math.random() * 0.1).toFixed(2));

      const telemetryData = {
        machine_id: MACHINE_ID,
        timestamp: new Date().toISOString(),
        status: "RUNNING",
        metrics: {
          motor_rpm: rpm,
          temperature_celsius: temperature,
          total_production_meters: parseFloat(totalMetersProduced.toFixed(2))
        }
      };

      console.log(`machine: ${MACHINE_ID} rpm: ${rpm} température: ${temperature} ...`);
      // Envoi des données vers le broker du PC Mint
      client.publish('bAcTechFlow-DataCenter/telemetry', JSON.stringify(telemetryData));
    }, 1000);

    console.log(`🏭 Machine [${MACHINE_ID}] initialisée et prête à produire !`);
  }
});

client.on('error', (err) => { console.error('❌ Erreur MQTT sur le simulateur :', err); });
```

3. Validation de la simulation en environnement natif

L'exécution simultanée des composants permet de valider le comportement du bus de messages en configuration standard.

L'invite de commandes ci-dessous montre la génération autonome et continue des trames de données de télémétrie brutes sur la machine de simulation de l'atelier :

En parallèle, la console du **PC Mint (Collecteur)** montre l'interception instantanée, le décodage des objets JSON et l'affichage des métriques centralisées prêtes pour l'archivage en base de données

```
Invite de commandes - node index.js
machine: USINE01-ATELIER02-MACHINE03 rpm: 766 température: 43.4 ...
machine: USINE01-ATELIER02-MACHINE04 rpm: 754 température: 42.6 ...
machine: USINE01-ATELIER02-MACHINE05 rpm: 768 température: 45.2 ...
machine: USINE01-ATELIER02-MACHINE01 rpm: 765 température: 42.7 ...
machine: USINE01-ATELIER02-MACHINE02 rpm: 754 température: 43.9 ...
machine: USINE01-ATELIER02-MACHINE03 rpm: 752 température: 45.4 ...
machine: USINE01-ATELIER02-MACHINE04 rpm: 761 température: 43.1 ...
machine: USINE01-ATELIER02-MACHINE05 rpm: 755 température: 43.2 ...
machine: USINE01-ATELIER02-MACHINE01 rpm: 755 température: 44.6 ...
machine: USINE01-ATELIER02-MACHINE02 rpm: 764 température: 44.3 ...
machine: USINE01-ATELIER02-MACHINE03 rpm: 755 température: 44.3 ...
machine: USINE01-ATELIER02-MACHINE04 rpm: 754 température: 43.7 ...
machine: USINE01-ATELIER02-MACHINE05 rpm: 759 température: 43.7 ...
machine: USINE01-ATELIER02-MACHINE01 rpm: 764 température: 43.5 ...
machine: USINE01-ATELIER02-MACHINE02 rpm: 768 température: 45 ...
machine: USINE01-ATELIER02-MACHINE03 rpm: 759 température: 44.5 ...
machine: USINE01-ATELIER02-MACHINE04 rpm: 759 température: 43.5 ...
machine: USINE01-ATELIER02-MACHINE05 rpm: 762 température: 44.6 ...
machine: USINE01-ATELIER02-MACHINE01 rpm: 763 température: 43.4 ...
machine: USINE01-ATELIER02-MACHINE02 rpm: 765 température: 44.6 ...
machine: USINE01-ATELIER02-MACHINE03 rpm: 756 température: 43.8 ...
machine: USINE01-ATELIER02-MACHINE04 rpm: 760 température: 44.4 ...
machine: USINE01-ATELIER02-MACHINE05 rpm: 763 température: 44.2 ...
machine: USINE01-ATELIER02-MACHINE01 rpm: 759 température: 45.4 ...
machine: USINE01-ATELIER02-MACHINE02 rpm: 752 température: 42.9 ...
machine: USINE01-ATELIER02-MACHINE03 rpm: 762 température: 42.7 ...

File Edit View Search Terminal Help
machine: USINE01-ATELIER02-MACHINE01 rpm: 754 température: 44.6 total production: 12612.84 état: RUNNING
machine: USINE01-ATELIER02-MACHINE02 rpm: 764 température: 44.5 total production: 12612.47 état: RUNNING
machine: USINE01-ATELIER02-MACHINE03 rpm: 755 température: 44.6 total production: 12613.49 état: RUNNING
machine: USINE01-ATELIER02-MACHINE04 rpm: 750 température: 43.7 total production: 12612.89 état: RUNNING
machine: USINE01-ATELIER02-MACHINE05 rpm: 761 température: 44.3 total production: 12612.47 état: RUNNING
machine: USINE01-ATELIER02-MACHINE01 rpm: 752 température: 45.2 total production: 12613.07 état: RUNNING
machine: USINE01-ATELIER02-MACHINE02 rpm: 767 température: 42.7 total production: 12612.74 état: RUNNING
machine: USINE01-ATELIER02-MACHINE03 rpm: 768 température: 45.4 total production: 12613.78 état: RUNNING
machine: USINE01-ATELIER02-MACHINE04 rpm: 759 température: 44.3 total production: 12613.14 état: RUNNING
machine: USINE01-ATELIER02-MACHINE05 rpm: 753 température: 43.7 total production: 12612.75 état: RUNNING
machine: USINE01-ATELIER02-MACHINE01 rpm: 767 température: 44.4 total production: 12613.26 état: RUNNING
machine: USINE01-ATELIER02-MACHINE02 rpm: 756 température: 45.2 total production: 12612.98 état: RUNNING
machine: USINE01-ATELIER02-MACHINE03 rpm: 760 température: 43.9 total production: 12614.07 état: RUNNING
machine: USINE01-ATELIER02-MACHINE04 rpm: 760 température: 44.7 total production: 12613.35 état: RUNNING
machine: USINE01-ATELIER02-MACHINE05 rpm: 754 température: 43.6 total production: 12613.01 état: RUNNING
machine: USINE01-ATELIER02-MACHINE01 rpm: 750 température: 45.2 total production: 12613.57 état: RUNNING
machine: USINE01-ATELIER02-MACHINE02 rpm: 758 température: 45.2 total production: 12613.23 état: RUNNING
machine: USINE01-ATELIER02-MACHINE03 rpm: 750 température: 43.6 total production: 12614.34 état: RUNNING
machine: USINE01-ATELIER02-MACHINE04 rpm: 751 température: 42.8 total production: 12613.58 état: RUNNING
machine: USINE01-ATELIER02-MACHINE05 rpm: 760 température: 45 total production: 12613.29 état: RUNNING
machine: USINE01-ATELIER02-MACHINE01 rpm: 769 température: 44.9 total production: 12613.84 état: RUNNING
machine: USINE01-ATELIER02-MACHINE02 rpm: 750 température: 43.6 total production: 12613.44 état: RUNNING
machine: USINE01-ATELIER02-MACHINE03 rpm: 766 température: 43.6 total production: 12614.61 état: RUNNING
machine: USINE01-ATELIER02-MACHINE04 rpm: 754 température: 45.3 total production: 12613.81 état: RUNNING
machine: USINE01-ATELIER02-MACHINE05 rpm: 757 température: 44.9 total production: 12613.45 état: RUNNING
machine: USINE01-ATELIER02-MACHINE01 rpm: 758 température: 44 total production: 12614.05 état: RUNNING
machine: USINE01-ATELIER02-MACHINE02 rpm: 758 température: 43.1 total production: 12613.71 état: RUNNING
machine: USINE01-ATELIER02-MACHINE03 rpm: 760 température: 43.4 total production: 12614.84 état: RUNNING
machine: USINE01-ATELIER02-MACHINE04 rpm: 762 température: 44.3 total production: 12614.03 état: RUNNING
machine: USINE01-ATELIER02-MACHINE05 rpm: 760 température: 43.6 total production: 12613.74 état: RUNNING
machine: USINE01-ATELIER02-MACHINE01 rpm: 766 température: 43.3 total production: 12614.31 état: RUNNING
machine: USINE01-ATELIER02-MACHINE02 rpm: 752 température: 45.4 total production: 12613.95 état: RUNNING
```

Flux des données des machines de l'atelier n° 1 simulées
par le PC Surface.

Flux des données des machines de l'atelier n° 2 simulées par
le PC Lenovo.

4. Évolution technologique : Supervision temps réel avec SignalR

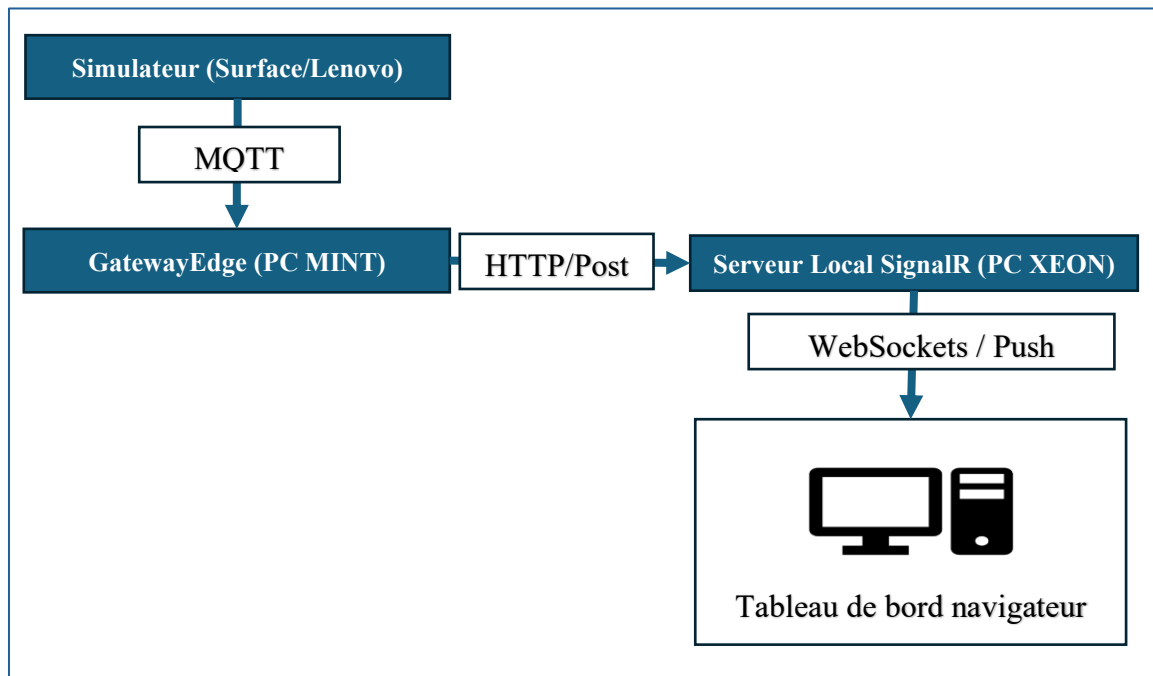
Pour transformer cette collecte de données console en une véritable interface de supervision industrielle interactive, l'architecture standard évolue en intégrant un serveur de communication temps réel basé sur ASP.NET Core SignalR.

Cette intégration permet de s'affranchir du rafraîchissement manuel des pages Web : dès qu'une métrique machine arrive sur le collecteur, elle est instantanément "poussée" vers les tableaux de bord des opérateurs de la salle de contrôle.

Pourquoi C# plutôt que Node.js :

Le C# est un langage compilé, fortement typé et extrêmement performant pour gérer les flux de données industriels surtout si on manipule des protocoles comme MQTT, OPC-UA ou de la communication directe avec les automates.

L'architecture du flux de données temps réel sera alors comme suit :



Architecture de flux des données temps réel.

A. Le Hub SignalR (C# - Côté Serveur Central sur PC Xeon)

Le Hub est le composant central qui gère les connexions des clients (les navigateurs des opérateurs) et diffuse les trames de télémétrie reçues. Pour intercepter les données, le secret réside dans une architecture API REST Web. L'application C# ne va pas chercher les données sur le Gateway Edge mais reste à l'écoute sur un port réseau (par exemple 8080) et attend que le Gateway Edge envoie des données via une requête http post. Pour que cela fonctionne, le serveur doit exposer un contrôleur API (un EndPoint).

Le code d'interception : TelemetryController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.SignalR;

namespace bAcTechFlow.FilialeServer.Services
{
    [ApiController]
    [Route("api/[controller]")]
    public class TelemetryController : ControllerBase
    {
        private readonly IHubContext<MonitoringHub> _hubContext;
        private readonly ILogger<TelemetryController> _logger;

        public TelemetryController(IHubContext<MonitoringHub> hubContext, ILogger<TelemetryController> logger)
        {
            _hubContext = hubContext;
            _logger = logger;
        }

        // POST api/telemetry
        // C'est ici que le Gateway Edge envoie ses données via HTTP POST
        [HttpPost]
        public async Task<ActionResult> ReceiveData([FromBody] TelemetryPayload payload)
        {
            if (payload == null || string.IsNullOrEmpty(payload.MachineId))
                return BadRequest("Payload invalide.");
        }
    }
}

```

```

// Diffusion immédiate vers tous les clients HTML via SignalR
await _hubContext.Clients.All.SendAsync("ReceiveData", new
{
    machine = payload.MachineId,
    rpm = payload.MotorRpm,
    temperature = payload.TemperatureCelsius,
    production = payload.TotalProductionMeters,
    etat = payload.Status,
    time = DateTime.Now.ToString("HH:mm:ss")
});
_logger.LogInformation("[REST] Télémétrie reçue et diffusée pour {Machine}", payload.MachineId);
return Ok(new { success = true });
}
}

// Modèle de données attendu depuis le collecteur JS
public class TelemetryPayload
{
    public string MachineId { get; set; } = "";
    public string Status { get; set; } = "RUNNING";
    public double MotorRpm { get; set; }
    public double TemperatureCelsius { get; set; }
    public double TotalProductionMeters { get; set; }
}
}

```

Le monitoring : MonitoringService.cs

```

using bAcTechFlow.Shared.Protos;
using Grpc.Core;
using Microsoft.AspNetCore.SignalR;
namespace bAcTechFlow.FilialeServer.Services
{
    // Héritage de la classe générée par votre fichier .proto
    public class MonitoringService : MonitoringStation.MonitoringStationBase
    {
        private readonly IMongoCollection<BsonDocument> _history;
        private readonly ILogger<MonitoringService> _logger;
        private readonly IHubContext<MonitoringHub> _hubContext;

        public override async Task<MeasureResponse> EnvoyerMesure(MeasureRequest request, ServerCallContext
context) {
            // Reconstitution de l'identifiant unique standardisé pour bAcTechFlow
            string machineId = $"{request.Machine}";

            //Diffusion instantanée en temps réel via SignalR vers les IHM de la salle de contrôle
            try {
                await _hubContext.Clients.All.SendAsync("ReceiveData", new {
                    machine = machineId,
                    temperature = request.ValeurT,
                    rpm = request.ValeurR,
                    production = request.ValeurP,
                    etat = request.Etat,
                    time = DateTime.Now.ToString("HH:mm:ss") });
                _logger.LogInformation("[SignalR] Diffusion temps réel réussie pour {Machine}.", machineId);
            }
        }
    }
}

```

```

catch (Exception ex) { _logger.LogError("[SignalR] ❌ ERREUR de diffusion : {Message}", ex.Message); }

// ici on implemente la partie de la logique métier pour traiter les données reçues, par exemple :
// sauvegarder dans une base de données, effectuer des calculs, etc.

return new MesureResponse { Succes = true };

}

}

```

Pour fonctionner on doit définir un fichier .proto (exemple bactec.proto) qui définit le prototype des données. La structure de ce fichier est la suivante et qui sera utilisé dans le MonitoringService.cs (using bAcTechFlow.Shared.Protos;)

```

public class Mesures
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; }
    public string StationId { get; set; } = null!;
    public double ValeurD { get; set; }
    public double ValeurT { get; set; }
    public double ValeurP { get; set; }
    public string Unite { get; set; } = null!;
    public string Etat { get; set; } = null!;

    [BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
    public DateTime DateEnregistrement { get; set; }
}

```

B. Interface de Visualisation (HTML5 / JavaScript - Salle de contrôle)

Les acteurs de la salle de contrôle utilisent une page web embarquant la bibliothèque client SignalR et une bibliothèque de traçage de graphiques (comme **Chart.js**). La mise à jour des graphiques se fait à la volée, sans latence. Le code html de la page peut être consulté dans le dossier wwwRoot.



exemple du tableau de bord.